Rapport de stage — M1 AM

---

# Exploration du paysage des Réseaux Booléens à Signe Pluripotents

---

Équipe BCM — Laboratoire TIMC, Grenoble

**Quentin Boyer**

Encadrants : Nicolas Glade, Laurent Trilling

Université Grenoble Alpes, Grenoble INP ENSIMAG

Mai-Juin 2026

**Résumé**

**Résumé.** [À rédiger en dernier.]

**Abstract.** [English version of the abstract.]

**Mots-clés :** Systèmes biologiques, réseaux booléens à signe (SBN), pluripotence, robustesse, évolutivité, Answer Set Programming, calcul parallèle.

# Table des matières

**1 Introduction** **4**
1.1 Motivations . . . . . 4
1.2 Positionnement dans les travaux de l'équipe . . . . . 5
1.3 Contributions de ce stage . . . . . 6

**2 Formalisme des Réseaux Booléens à Signe** **7**
2.1 Structure et dynamique . . . . . 7
2.2 Fonctions booléennes à seuil (SBF) . . . . . 7
2.3 Statistiques, propriétés, métriques . . . . . 10
2.4 Métagraphes dynamiques et structurels, règles de mutation . . . . . 10
2.5 Quel paysage des réseaux explore-t-on ? . . . . . 12

**3 Méthodes et outils** **13**
3.1 Correction . . . . . 13
3.2 Optimisation . . . . . 15
3.3 Observation . . . . . 15
3.4 Usage des grands modèles de langage en recherche . . . . . 15
3.5 Exploration Randomisée : MCSBN . . . . . 16

**4 Postérité et résultats** **16**
4.1 Guide de génération des CSV . . . . . 16
4.2 Guide d'utilisation des HTML . . . . . 16
4.3 Observations et limites . . . . . 16

**5 Conclusion, travaux inachevés et ouverture** **17**

**A Guide d'utilisation des programmes** **20**
A.1 Organisation du dépôt et installation . . . . . 20
A.2 Format des CSV . . . . . 21
A.3 Programmes ASP d'énumération . . . . . 22
A.4 JPPipeline : énumération parallèle et statistiques . . . . . 23
A.5 MCSBN : exploration Monte-Carlo . . . . . 24
A.6 Scripts annexes et programmes historiques . . . . . 26

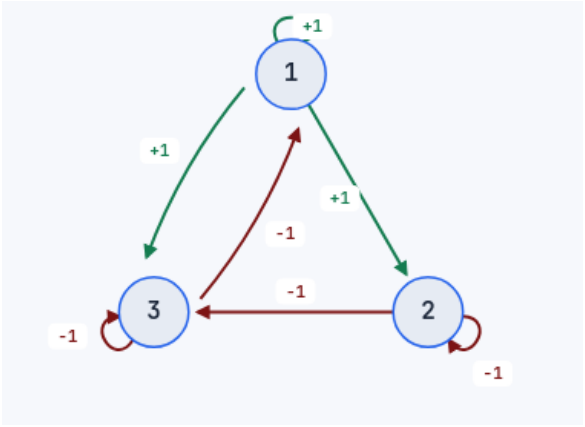
# 1 Introduction

## 1.1 Motivations

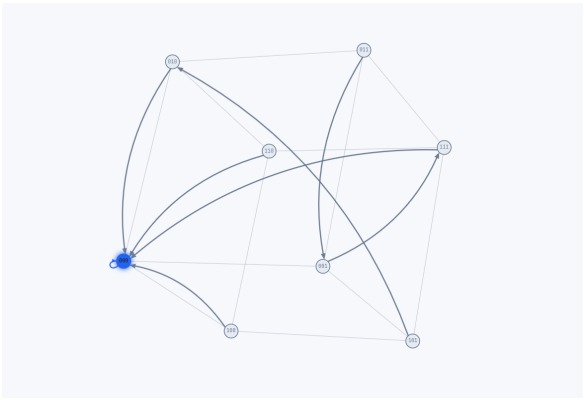
Un des intérêts de la modélisation des systèmes biologiques au sens large et à toute échelle (que ce soient des écosystèmes,des cellules, l'expression génétique, des relations symbiotiques, des neurones...) est qu'elle permet suffisamment de simplifications et d'abstractions pour pouvoir isoler (donc clarifier) et forger des concepts sur lesquels on peut facilement travailler (au sens mathématique) et dont on peut évaluer la pertinence (ce qui est un objectif en fin de compte, forger des concepts pertinents pour que ceux qui viennent après s'en emparent). L'étude in vivo (ou juste du vivant ?) ne peut généralement qu'entrapercevoir ces mêmes concepts qu'au milieu du fouillis des particularités biologiques, et encore, souvent au travers du prisme de la vie telle qu'on la connaît (prisme duquel on voudrait s'affranchir s'il est question d'origine du vivant, voire d'exobiologie), là où le modèle ouvre la voie à l'exhaustivité et à la généralisation (on a bien dit ouvre la voie, pas résout le problème). C'est donc dans ce cadre que nous nous plaçons : l'exploration des horizons d'un modèle (celui des Signed Boolean Network SBN) pour observer les contraintes qu'il impose et les lois que celles-ci font émerger (développer l'idée que contraindre permet ?). Il n'est donc pas question ici d'évaluer la validité biologique du modèle (même si nos réflexions à ce sujet guident nos choix) mais de bien modestement (j'ai deux mois, ce n'est pas énorme) en exploiter le potentiel conceptuel (et l'intérêt porté à cette exploitation est en quelque sorte une mesure de sa validité (i.e. utilité) conceptuelle).

La caractéristique du vivant qui sert de point de départ à ce travail est la pluripotence, c'est-à-dire la capacité d'un système unique à présenter différents comportements qui eux-mêmes correspondent à d'autres systèmes similaires mais potentiellement plus "simple", on dit que le système principal les "simule". Typiquement l'expression du code génétique (d'où proviendra souvent par analogie le vocabulaire que nous emploierons) dont la spécification des cellules est un bon exemple, des poils de chat court ou long selon la saison, la génération des pensées par nos neurones... [À fournir et préciser].

Le modèle exploré ici pour formaliser ces concepts est donc celui des Réseaux Booléen à Signe $SBN$, formalisé en section [X.] et ce qui nous y intéresse particulièrement, c'est l'interprétation des $n$ nœuds du réseau comme une STRUCTURE (ou un programme, un code, un plan...) possédant différent états, et dont l'évolution déterministe génère une DYNAMIQUE (ou une exécution, une expression, une construction...) dans l'espace du graphe de transition de ces états (hypercube à n dimensions).



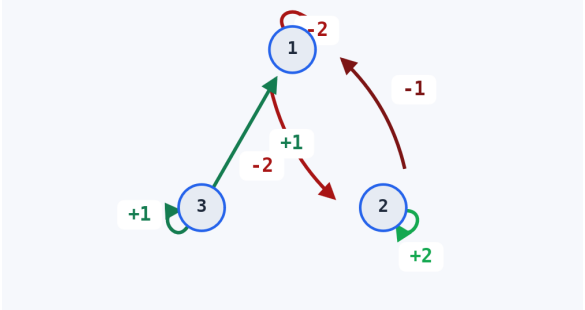

(a) Une structure quelconque



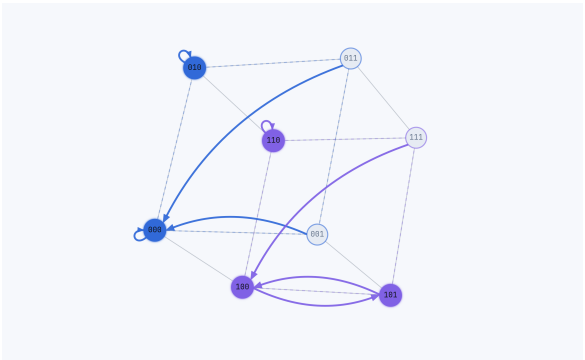
(b) Sa dynamique

FIGURE 1 – Un réseau et son graphe de transition en dimension 3.

Le concept de pluripotence est volontairement suffisamment général et flou pour pouvoir capter les différentes interprétations que le vivant peut en faire. Elle pourrait ainsi prendre sens dans le formalisme des SBN comme la présence de sous-dynamiques stables correspondant à des SBN sur des supports pouvant être encodé par une dimension inférieure. Mais ce n'est pas la seule manière, on pourrait aussi (et c'est plutôt ce qui est fait ici comme première approche plus simple) rechercher des sous-dynamiques disjointes sur des supports identifiés par un unique noeud (dit "de contrôle"). Le choix des mots est important et il nous appartient d'être clair sur l'aspect de la pluripotence que l'on met en évidence. [Cf. Quel paysage explore-t-on ?].




(a) Une structure pluripotente. Le noeud 3 est un noeud de contrôle.



(b) Sa dynamique possède des hyper-faces disjointes (marquées par différentes couleurs).

FIGURE 2 – Un réseau pluripotent et son graphe de transition.

## 1.2 Positionnement dans les travaux de l'équipe

Le présent travail n'est pas isolé, il s'intègre dans une réflexion plus large menée de longue date par Nicolas Glade sur le vivant, et à laquelle ont participé (et participe) de nombreux chercheurs et étudiants (souvent dans l'équipe BCM du TIMC). Consulter leurs écrits s'est avéré très instructif (le lecteur y est largement invité) et aura permis en

quelque sorte de "Se tenir sur les épaules des géants et voir plus loin, voir dans l'invisible, à travers l'espace et à travers le temps." - Jean-Claude Ameisen.

En voici une liste (non exhaustive, mais rassemblant les principaux).

- HDR de Nicolas Glade : la bible donnant le cadre général [Cf. tout].

- Thèse de Rémi Segretain : réflexions générales sur les SBN et développement de la Java Parallel Pipeline (JPP) [Cf. partie technique]

- Rapports d'Ailing Bonnet et de Rémy Defossez : approfondissement des travaux de Rémi [Cf. partie calcul de propriétés].

- Rapport de Danwen Wu : Démonstration mathématique de l'équivalence entre l'existence de dynamiques séparées sur deux hyper-faces disjointes et de contraintes sur les poids du réseau [Cf. partie Méthodologie d'exploration].

- Programmes de Laurent Trilling : le point de départ de ce travail, l'énumération en Answer Set Programming (ASP) des réseaux pluripotents sous nœud de contrôle via la dynamique [Cf. partie Méthodologie d'exploration].

Pour ce qui est des travaux encore en cours au moment de la rédaction, Corentin Lacaze s'occupe de la simulation d'agents dont les comportements sont intégralement générés par des SBN. L'objectif étant de les mettre en compétition pour observer des phénomènes évolutifs et comprendre comment cela se traduit en termes de SBN. La complémentarité entre l'aspect théorique du présent travail d'un côté et l'aspect expérimental des simulations de l'autre aura alimenté nos échanges qui nous auront permis l'un l'autre de nous guider mutuellement.

[Image issue des simulations]

## 1.3 Contributions de ce stage

L'apport principal de ce travail est la recherche de la pluripotence dans les SBN. L'enjeu principal a donc été d'abord d'énumérer efficacement les SBN qui nous intéressent, la définition de ce qui nous intéresse étant souvent guidé par l'exigence d'efficacité (recherche de symétries). L'efficacité mémoire et temporelle est cruciale, les espaces de recherche explosent combinatoirement avec le nombre de noeuds du réseau et avec nos méthodes, il n'est en pratique pas raisonnable d'espérer l'exhaustivité à partir de 4 noeuds, où même d'avoir des données représentatives à partir de 5 noeuds [Cf table des échelles]. Une bonne partie du temps de travail a donc été dédié à l'optimisation du code pour essayer de récolter le maximum d'information avec un minimum de calculs.

Dans un second temps l'objectif a été de créer des visualisations utiles et pratiques, à la fois pour vérifier la justesse de nos programmes, mais aussi pour apprécier intuitivement l'espace des réseaux à travers différents angles et différentes métriques. Cela se matérialise par les trois scripts hmtl/javascript présents à la racine du git, permettant tous les trois de charger des fichiers CSV générés indépendamment et d'en afficher de manière concise

le contenu.

- sbn_viz.html : structures et dynamiques détaillés.

- sbn_scatter.html : nuages des dynamiques placées selon différentes métriques.

- metagraphe.html : nuages des dynamiques placées les unes par rapport aux autres selon des règles de mutations.

[Cf. partie postérité pour un usage détaillé]

Ce stage aura aussi donné l'occasion de rédiger un article (i.e. plus ou moins ce papier) synthétisant un bout de chemin parcouru sur la longue route qu'est la réflexion sur les SBN ?

# 2 Formalisme des Réseaux Booléens à Signe

## 2.1 Structure et dynamique

[Cf. rapport Ailing]

## 2.2 Fonctions booléennes à seuil (SBF)

[Cf. rapport Ailing]

Commentaire sur les permutations des poids des SBF et la non prise en compte d'une arête non existante (chemin à exploiter). Ceci explique les différences entre cette table d'échelle et celles qui ont étés faits précédemment.

TABLE 1 – Ordres de grandeur par dimension $n$. **SBF** $= |\mathcal{T}_n|$ (chambres, BFS exact) ; **orb.** $= |\mathcal{T}_n/S_n|$ (réduction par permutation des nœuds) ; **SBN** $= |\mathcal{T}_n|^n$ (*produit de SBF* : tout $n$-uplet de SBF est réalisable, cf. indépendance des colonnes) ; **PTBN** $= |\mathcal{T}_n|^n - (|\mathcal{T}_n|-1)^n$ (SBN admettant $\geq 1$ nœud de contrôle, i.e. une colonne $f_c = \pi_c$ ; forme close validée par énumération) ; $B = w_{\text{bound}}(n)$ (ILP, pire cas) ; **Poids** $= (2B+1)^{n^2}$ (espace syntaxique des matrices) ; **vec.** vecteurs de décomposition (couverts par `setup_btree`) ; **btree** arbres non étiquetés $A(d) = 1+A(d-1)^2$ (`treeinsolver`) ; **tagged** arbres étiquetés par nœud de contrôle $T(m) = 1 + m\,T(m-1)^2$ (`taggedtree`).

| $n$ | SBF | orb. | SBN | PTBN | $B$ | Poids | vec. | btree | tagged |
|---|---|---|---|---|---|---|---|---|---|
| *form.* | $\lvert\mathcal{T}_n\rvert$ | $\lvert\mathcal{T}_n/S_n\rvert$ | $\lvert\mathcal{T}_n\rvert^n$ | $\lvert\mathcal{T}_n\rvert^n-(\lvert\mathcal{T}_n\rvert-1)^n$ | $w_{\text{b}}(n)$ | $(2B+1)^{n^2}$ | $\lvert H_n\rvert$ | $1+A_{n-1}^2$ | $1+n\,T_{n-1}^2$ |
| 1 | 2 | 2 | 2 | 1 | 1 | 3 | 2 | 2 | 2 |
| 2 | 6 | 4 | 36 | 11 | 1 | 81 | 4 | 5 | 9 |
| 3 | 32 | 10 | $3.28{\cdot}10^4$ | 2977 | 2 | $1.95{\cdot}10^6$ | 10 | 26 | 244 |
| 4 | 370 | 36 | $1.87{\cdot}10^{10}$ | $2.02{\cdot}10^8$ | 3 | $3.32{\cdot}10^{13}$ | 36 | 677 | $2.38{\cdot}10^5$ |
| 5 | 11292 | 224 | $1.84{\cdot}10^{20}$ | $8.13{\cdot}10^{16}$ | 5 | $1.08{\cdot}10^{26}$ | 202 | $4.58{\cdot}10^5$ | $2.84{\cdot}10^{11}$ |
| 6 | $1.07{\cdot}10^6$ | 3024 | $1.47{\cdot}10^{36}$ | $8.26{\cdot}10^{30}$ | 9 | $1.08{\cdot}10^{46}$ | 1828 | $2.10{\cdot}10^{11}$ | $4.82{\cdot}10^{23}$ |

Les colonnes **Poids** et **SBN** explosent (respectivement $(2B+1)^{n^2}$ et $|\mathcal{T}_n|^n$) : la force brute est intenable dès $n = 4$. Les trois dernières colonnes donnent les trois leviers qui rendent la montée en dimension possible : réduction par $S_n$ (facteur jusqu'à $n!$), restriction au sous-espace pluripotent (*vec.*, *btree*, *tagged* $\ll$ SBN), et énumération par arbre-dans-le-solveur plutôt que par grille de poids. Les trois familles d'arbres sont emboîtées, vec. $\leq$ btree $\leq$ tagged, et correspondent à trois outils de complétude croissante : `setup_btree` (une orientation par vecteur, *incomplet*) $\subset$ `treeinsolver` (toutes orientations) $\subset$ `taggedtree` (étiqueté, sans permutation).

**Borne des poids par programmation linéaire en nombre entier.** On note $\mathcal{T}_n = \{f_w : w \in \mathbb{Z}^n\}$ l'ensemble des SBF réalisables en dimension $n$.

Pour une SBF réalisable $f$, posons $P = \{\mathbf{x} : f(\mathbf{x}) = 1\}$ et $N = \{\mathbf{x} : f(\mathbf{x}) = 0\}$. La borne de poids minimale réalisant $f$ est $W(f) = \min\{\|w\|_\infty : w \in \mathbb{Z}^n,\ f_w = f\}$. Comme $\sum_i w_i x_i \in \mathbb{Z}$ pour $w$ entier, la condition stricte $\sum_i w_i x_i > 0$ équivaut à $\sum_i w_i x_i \geq 1$, ce qui permet d'écrire le problème comme un programme linéaire en nombres entiers :

$$
\begin{aligned}
W(f) = \min_{w\in\mathbb{Z}^n,\ B\in\mathbb{Z}_{\geq 0}} \quad & B \\
\text{s.c.} \quad & \textstyle\sum_i w_i x_i \geq 1 \qquad \forall \mathbf{x} \in P, \\
& \textstyle\sum_i w_i x_i \leq 0 \qquad \forall \mathbf{x} \in N, \\
& -B \leq w_i \leq B \quad \forall i \in \{1,\dots,n\}.
\end{aligned}
\tag{1}
$$

Les deux premières familles de contraintes caractérisent $f$ ; $-B \leq w_i \leq B$ linéarise $B = \|w\|_\infty$.

La borne recherchée, celle qui permet d'exprimer *toutes* les SBF de dimension $n$, est alors le pire cas sur $\mathcal{T}_n$ :

$$
\boxed{\max_{f_w\in\mathcal{T}_n} W(f)} \tag{2}
$$

c'est-à-dire le plus petit $B$ tel que tout $f_w \in \mathcal{T}_n$ soit réalisable avec $\|w\|_\infty \leq B$.

Il ne reste donc plus qu'à énumérer toutes les SBF et résoudre l'ILP ci-dessus pour chacune. Enumérer les $2^{2^n}$ fonctions booléennes pour ne selectionner que les réalisables n'est pas raisonnable pour $n \geq 5$, il faut donc trouver une autre manière. Une idée est de passer par les poids, il suffirait de trouver *une* borne minimale des poids garantissant l'obtention de toutes les SBF, mais il n'en a pas été trouvé de suffisamment raisonnable pour le calcul. L'idée retenue a donc été d'explorer l'espace des SBF par un parcourt (en largeur) de graphe, en considérant que deux SBF sont voisines si on peut passer de l'une à l'autre par changement d'un unique bit, i.e. en traversant un unique hyper-plan de $\mathbb{R}^n$ dans l'espace des poids. [Cf la thèse de Rémi, les parties 3 et 4 présentent une approche différente d'énumération que celle évoquée ici, mais ses illustrations permettent d'y voir plus clair]

**Complétude du parcours.** Cette relation de voisinage est exactement l'adjacence des *chambres* de l'arrangement des $2^n$ hyperplans $H_{\mathbf{x}} = \{w : \sum_i w_i x_i = 0\}$ dans l'espace des poids : sur chaque chambre (région ouverte délimitée par ces hyperplans), le signe de $\sum_i w_i x_i$ est constant pour tout $\mathbf{x}$, donc $f_w$ y est constante. $\mathcal{T}_n$ est donc exactement l'ensemble des chambres, et deux chambres voisines (séparées par une seule facette) donnent deux SBF qui ne diffèrent que d'un bit.

Le graphe des chambres est connexe : $\mathbb{R}^n$ (l'espace complet, avant de retirer les hyperplans) est connexe, donc deux poids quelconques $w, w'$ sont reliés par un chemin ; quitte à le perturber légèrement, ce chemin ne traverse jamais deux hyperplans au même instant (cas de mesure nulle), donc il les traverse un par un, chaque traversée faisant passer à une chambre voisine. La suite des chambres ainsi traversées relie celle de $w$ à celle de $w'$ : le graphe est connexe.

Un parcours en largeur partant d'une seule SBF réalisable (par exemple $f = 0$, obtenue avec $w = 0$) et ne testant, à chaque étape, que la réalisabilité du voisin à un bit de distance (simple faisabilité LP) visite donc nécessairement toute $\mathcal{T}_n$ : la connexité garantit qu'aucune SBF n'est hors d'atteinte, contrairement à une recherche par force brute sur les poids où une SBF exigeant de gros poids peut être manquée sans avertissement.

**Réduction par symétrie.** Deux SBF liées par une permutation des $n$ coordonnées ont la même borne $\|w\|_\infty$ (permuter les poids ne change pas leur norme), il suffit donc d'explorer un seul représentant par orbite sous $S_n$ . Cela ne coûte rien en complétude : si $r$ est le représentant exploré et $\sigma \cdot r$ un autre représentant de la même orbite, retourner le bit $k$ de $\sigma \cdot r$ revient, après application de $\sigma^{-1}$, à retourner le bit $\sigma^{-1}(k)$ de $r$ ; comme $\sigma^{-1}$ est une bijection sur les positions, les $2^n$ voisins de $r$ (une fois ramenés à leur représentant canonique) donnent déjà toutes les orbites voisines, quel que soit le représentant de départ choisi.

TABLE 2 – Les 6 SBF distinctes en dimension 2. $f(\mathbf{x}) = 1$ ssi $\sum_i w_i x_i > 0$, avec $w_i \in \{-1, 0, 1\}$.

| $(w_0, w_1)$ repr. | $f(0,0)$ | $f(1,0)$ | $f(0,1)$ | $f(1,1)$ | Expression |
|---|---|---|---|---|---|
| $(-1, -1)$ | 0 | 0 | 0 | 0 | $0$ |
| $(1, -1)$ | 0 | 1 | 0 | 0 | $x_0 \wedge \neg x_1$ |
| $(-1,\ 1)$ | 0 | 0 | 1 | 0 | $\neg x_0 \wedge x_1$ |
| $(1,\ 0)$ | 0 | 1 | 0 | 1 | $x_0$ |
| $(0,\ 1)$ | 0 | 0 | 1 | 1 | $x_1$ |
| $(1,\ 1)$ | 0 | 1 | 1 | 1 | $x_0 \vee x_1$ |

## 2.3 Statistiques, propriétés, métriques

Donner des mesures fait parties des moyens que l'on a pour former des groupes, des relations d'équivalence, des relations d'ordre, bref pour se faire des intuitions sans avoir à éplucher l'intégralité de l'information contenue dans chaque réseau (ce qui est impossible, donc on fait une sorte de modèle dans le modèle). C'est pourquoi en plus d'énumérer les réseaux, nous calculons des métriques associés qui sont bien utiles pour visualiser les réseaux dans différents espaces. Le choix des statistiques calculées est motivé par les travaux de [Rémi] et [Ailing], mais reviennent aussi beaucoup au papier de [Wagner]. Un axe d'amélioration du présent travail serait les calculs d'autres statistiques, ce qui permettrait d'élargir l'horizon observable (dans les limites du passage à l'échelle nécessaire).

Explication dans les grandes lignes en quoi le papier de Wagner nous intéresse :

- Robustesse.
- Évolutivité.

[Scatter robustesse evolutivité]

-> confirmation des résultats de Wagner.

Autres métriques intéressantes :

- Complexité au sens de rareté.
- Bassins d'attraction, attracteurs.
- Calcul sur des poids minimisés (non fait).

## 2.4 Métagraphes dynamiques et structurels, règles de mutation

Une approche proposée dans la thèse de Rémi (et poursuivie par Rémy et Ailing) a été de construire des méta-graphes dans lesquels deux dynamiques sont reliées si elles sont voisines en termes de distances (pouvant répondre à différentes définitions) et dont la position est calculée par un algorithme de type Force-directed.

[figure métagraphe issue du rapport d'Ailing]

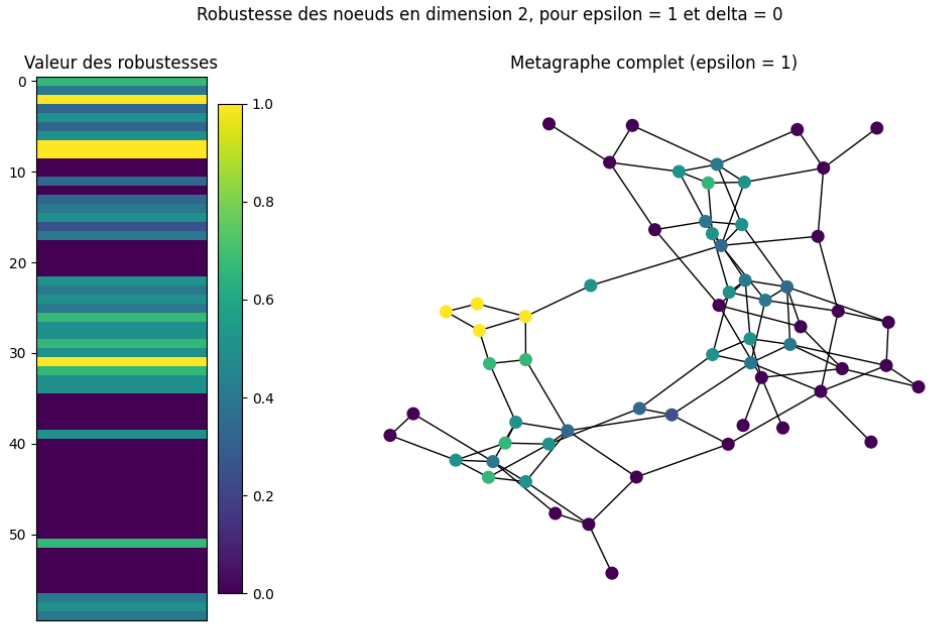



FIGURE 3 – Exemple d'un métagraphe sur les réseaux de dimension 2. Accompagné d'une coloration des noeuds

À noter qu'ici, les noeuds du graphe sont des dynamiques,

Il a été essayé de reproduire les mêmes méta-graphes en réutilisant les définitions de distances, en particulier la distance DA agrégeant les changements des cycles limites. Mais comme le calcul exhaustif de ces distances tel quels ne pouvait être mis à l'échelle pour la dimension 3 qu'aux prix d'efforts conséquents (de l'ordre de 1e10 paires), et est difficilement concevable en dimension 4 (de l'ordre de 4e20 paires), il n'a pas été poursuivi. Il est sans doute possible de trouver des manières de contourner cette difficulté qui n'est que technique, et de construire des méta-graphes similaires sémantiquement. En tout cas cette piste (qui existe) n'a pas été exploré plus avant ici.

Cependant l'idée de construire des méta-graphes Force-directed n'a pas été totalement mise de côté, nous nous sommes juste plus modestement limités à des méta-graphes structurels (ou les dynamiques sont liées si elles possèdent des structures voisines). Cela nous permet notamment de faire un lien direct avec les simulations de Corentin Lacaze, car les règles de mutations qui dirige l'évolution des agents dans ce contexte sont des relations de voisinages entre structures. L'exploration contrainte par l'évolution des agents se traduit alors par un chemin (avec de potentiels embranchements, donc un arbre) dans le méta-graphe, et cela permet de poser des questions du type "Est-ce que certaines propriétés des dynamiques favorise le passage des chemins", "Certaines zones sont-elles plus viables que d'autres ? C'est-à-dire est-ce que les chemins préfèrent certaines zones ?" "Pour arriver à un phénotype cible, est-ce que plusieurs chemins sont possibles, si oui en quoi sont-ils différents ?" .

[Figure métagraphe issu du html]

La règle de mutation considérée en premier a donc été simplement celle d'un changement de poids quelconque (dans les limites de la borne inf des poids permettant d'exprimer toutes les dynamiques) sur un seul arc du réseau. Il serait intéressant de proposer d'autres règles de mutation pour observer les différences de résultats selon le prisme qu'elles constituent, et notamment des règles pouvant faire varier la dimension des réseaux : l'ajout ou la suppression de noeuds permettrait de prendre du recul par rapport à la dimension (qui reste un carcan solide dans cette étude).

[Insérer équation "Mathématiques" définissant une arête du métagraphe]

[Cf partie utilisation de ma Pipeline pour modifier ces règles de mutations]

## 2.5 Quel paysage des réseaux explore-t-on ?

Les différents aspects de la pluripotence :

- Noeuds de contrôle -> vecteurs de décomposition complet -> fichiers undercontrol.

[Formalisation Cf. rapport Danwen]

[Figure issue de `sbn_viz` montrant une dynamique décomposable dans ce sens là.]

- Formalisme des sous réseaux -> fichiers PSBN/PTBN.

[Formalisation : contraintes d'existence de réseaux simulés]

[Figure issue de `sbn_viz` montrant une dynamique non décomposable en SBN mais décomposable en TBN]

- Contrainte suplémentaire -> fichiers PSBNI (I pour inhibiteurs).

[Formalisation : restriction du domaine des poids]

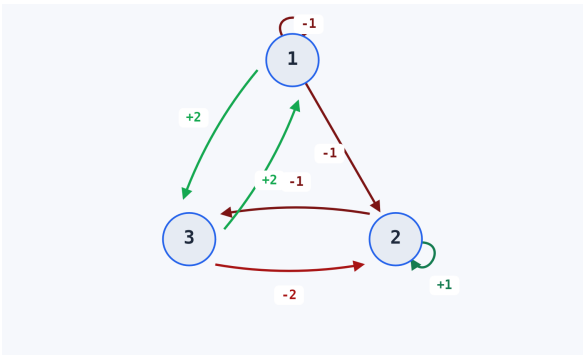
[Figure issue de sbn_viz montrant un PSBNI]

- La partialité -> vecteurs de décomposition partiel -> non explorée.

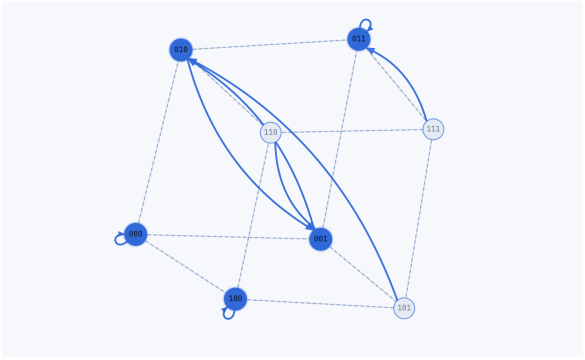
[Figure issue de sbn_viz montrant les supports stables et un vecteur de décomposition partiel]

- Noeuds contrôlés -> offuscation -> non explorée.

[Figure montrant une structure organisée en 2 noeud PC et 1 noeud PO, dynamique associée]




(a) Une structure pluripotente sous réseau de contrôle de dimension 2. Ici les noeuds 1 et 3 sont de contrôle.



(b) Sa dynamique possède des supports disjoints correspondant à l'extension d'une dimension de la dynamique de sa partie contrôle.

FIGURE 4 – Un réseau pluripotent sous réseau de contrôle.

# 3 Méthodes et outils

Comme souvent dans l'écriture de code, les enjeux des phases de développement ont toujours été dans un premier temps d'arriver au résultat attendu (la correction) puis d'optimiser pour passer à l'échelle (la terminaison, idéalement rapide). Mais comme la correction dans notre cas nécessite surtout de bien savoir ce que l'on cherche, et que pour cela, il faut avoir des visualisations efficaces, les étapes de correction, d'optimisation et d'évaluation des rendus se sont succédé en cycles. Les sections suivantes ne reflètent donc pas nécessairement fidèlement la chronologie du développement.

## 3.1 Correction

L'objectif originel du stage était de prendre en main le language Answer Set Programming (ASP) pour parvenir à comprendre et améliorer les programmes existant [Cf. Programmes Laurent dans le git].

Explication/Promo ASP

- Clingo [Cf. potassco etc]
- Clingcon [Cf. lien du git]
- ApproxAsp [Cf. lien du git]

Explication du programme de Laurent

- travail sur la dynamique [Cf. le papier pas vraiment fini de Laurent]
- Systématisation et export en csv via le script python `export_decomposition.py`. Visualisation via `sbn_viz` pour mieux apprécier.
- Génération des réseaux pluripotents à 1 noeud de contrôle, de sous-réseau SBN, d'arbre de décomposition complet fixé, avec hyper-faces distinctes. Calcul lourd et peu

scalable (utilisable en temps et ressources raisonnables que jusqu'en dimension 3).

[Figure screenshot de `sbn_viz` sur le fichier des 189 $\langle 0, 2, 0, 0\rangle$]

[Lien vers le fichier utilisable]

Le rapport de Danwen prouvant qu'il est possible de caractériser cette famille de réseaux en ne travaillant que sur les poids, c'est dans cette direction que nous sommes partis, aboutissant au fichiers :

- `PTBN_undercontrol.lp` permettant de générer des réseaux pluripotents à 1 nœud de contrôle, et d'arbre de décomposition fixé. (Et de sous-réseaux quelconques, donc TBN à cause du contrôle).

[Figure screenshot de `sbn_viz` sur le fichier $\langle 0, 2, 0, 0\rangle$ sélectionnant un non PSBN]

[Lien vers un fichier utilisable]

- `PSBN_undercontrol.lp` extension du fichier précédent permettant de préciser que les sous-réseaux sont bien des SBN.

[Figure screenshot de `sbn_viz` sur le fichier $\langle 0, 2, 0, 0\rangle$]

[Lien vers un fichier utilisable]

La propriété d'avoir différentes hyper-faces nous a semblé peu essentielle [préciser] au regard des rendus que l'on voulait afficher par la suite et a donc été laissée de côté. (`PSBN_distinct_faces_HS.lp` a quand même été développé dans l'idée de confirmer que nos résultats correspondaient à ceux des programmes initiaux, ce qui a été le cas pour certaines configurations : les 189 $\langle 0, 2, 0, 0\rangle$ en $d = 3$ notamment).

- `PSBN_undercontrol_undecomposable_beyond.lp` et `PSBN_undercontrol_undecomposablePBN_b`
Question de décomposition fines. Échecs.

[donner une table des échelles de décomposition pour pourquoi ce n'est pas si important]

[Figure illustrant un PSBN trop décomposable sur $\langle 0, 2, 0, 0\rangle$ qu'on voudrait ne pas énumérer]

[Lien vers un fichier utilisable]

- `PTBN_undercontrol_treeinsolver.lp` prenant en compte n'importe quel arbre de décomposition à vecteur de décomposition fixé. Prise en compte des permutations sur les noeuds du réseau principal, mais sur ceux des sous-réseaux.

[Figure screenshot de `sbn_viz` sur le fichier $\langle 0, 2, 0, 0\rangle$ montrant une orientation et arbre différent de ceux vu jusque là (sur un décomposable uniquement TBN)]

[Lien vers un fichier utilisable]

- `PSBN_undercontrol_treeinsolver.lp` de la même manière qu'avec les fichiers `undercontrol` simple, extension du fichier précédent permettant de préciser que les sous-réseaux sont bien des SBN.

[Figure screenshot de `sbn_viz` sur le fichier $\langle 0, 2, 0, 0\rangle$ cette fois ci sur un décomposable SBN]

[Lien vers un fichier utilisable]

- `PTBN_undercontrol_taggedtree.lp` prise en compte des permutations des noeuds des sous-réseaux (i.e. d'un arbre de décomposition dont les branchements sont étiquetés par le numéro du noeud de contrôle associé).

[Figure screenshot de `sbn_viz` sur le fichier $\langle 0, 0, 2, 4 \rangle$ montrant ]

[Lien vers un fichier utilisable]

[explication du fonctionnement précis du code en annexe ?]

## 3.2 Optimisation

- Base ASP : contraintes sur les poids plus efficaces que les contraintes sur la structure. Parallélisation possible directement en ASP, mais généralement peu efficace [donner des chiffres].

-> Répertoire JPPipeline du git. (Java)

- JPPipeline : parallelisation à plus au niveau permettant de mieux tirer partie des différent coeurs de la machine, et de mieux gérer l'utilisation mémoire.

- Le fonctionnement général.

- Ma pipeline.

[Donner une image globale avec des instances à chaque fois].

- L'énumération avec garde-fou.

[Zoom sur la partie énumération]

- utilisation ?

- post processing python relativement abandonné ici.

- Utilisation machine HPC [Donner les spécs de la machine et des exemples de gains]

## 3.3 Observation

-> fichiers html à la racine du git.

- Beaucoup de questions de Front : lisibilité (palette de couleur, styles des points, fonctions interactives, ordre d'affichage), fluidité (webGL, parsing en streaming, rafraîchissement partiel), ergonomie (parsing des différentes conventions de CSV mises en place pendant le développement)

- Mais aussi beaucoup de questions de fonds : Comment organiser la légende ? Quelles statistiques afficher ? Comment donner le plus d'information possible ? Barres d'erreur ? Quelle décomposition mettre en valeur ?

## 3.4 Usage des grands modèles de langage en recherche

- Possibilités permises, rapidité de développement.

- Usage différencié selon les parties du projet.

- Claude code en version pro. Utilisation relativement naïve, il est probablement possible de l'utiliser encore plus efficacement.

### 3.5 Exploration Randomisée : MCSBN

-> répertoire MCSBN du git. (python et C)

Tentative en véritable vibe code (autorisation d'écriture sans demande de permission) d'énumération sans passer par l'asp pour voir 1. Si c'est plus efficace 2. Quelles en sont les limites.

- énumération brute puis post processing -> on ne récupère quasiment que des non décomposés.

- énumération d'un coeur pluripotent puis de sa frontière en marche aléatoire (`core_then_walk.py`) -> illustre bien que la manière de générer est déterminante.

- utilisation de DRGraph pour précalculer le layout Force-directed rapidement.

## 4 Postérité et résultats

Un des objectifs du projet étant son utilisation par autrui, j'essaie de présenter ici les moyens de (~~s'amuser avec~~) s'en servir sans avoir à s'y plonger trop avant.

### 4.1 Guide de génération des CSV

Utilisation de la pipeline :

- Changer les arguments [donner pseudo doc en annexe] : avec VScode utiliser le launch.json ou alors par le terminal.

- Changement d'énumérateur asp : à implémenter dans `JPPipeline/src/asp` en gardant les mêmes noms de prédicat (ou inspecter les regexp mais ce n'est pas recommandé).

- Changer le garde-fou hard-codé.

- Changer la limite de process d'énumération en parallèle hard-codé.

- Changement de statistiques : à implémenter dans le processeur `JPPipeline/src/java_src/PSBN/S`

- Compresser les résultats via gzip (les htmls gèrent les .gz)

Utilisation de MCSBN :

### 4.2 Guide d'utilisation des HTML

### 4.3 Observations et limites

Dans un premier temps, il est rassurant d'observer que nos graphiques confirment les résultats de [Wagner]. Les SBN possédant la même propriété que les séquences d'ARN

sur les fréquences d'apparitions de phénotypes, on retrouve bien une corrélation positive entre évolutivité et robustesse phénotypiques.

[Figure graphique scatter EP/RP en plusieurs dimensions (+ p-valeur)]

Malgré nos limitations à des dimensions faibles, il est possible d'observer des tendances dont l'explication pourrait éventuellement permettre de généraliser aux dimensions supérieures. [Insérer des graphiques avec des jolies flèches paint pour chaque assertion]

- La décomposition d'un SBN en sous-dynamiques distinctes est déterminante dans le calcul de ses autres propriétés. (La pluripotence n'est pas décorrélée de tout)

- Les propriétés d'un PSBN semblent être très liés à la dimension de sa plus grande hyper-face non décomposable. Notamment la variance de la robustesse génotypique pour chaque phénotype (Robustness_std)

- [importance de SBF particulières pour former des groupes dans le graphique variance/moyenne de robustesse]

Dans le contexte de mutations sur les poids [Cf. règles de mutations] :

- Les PSBN sont plutôt proches les uns des autres, notamment au sein d'une même orientation (élément d'une orbite de permutation). Les PSBN les plus décomposés (que les permutations affectent moins[illustration]) font en quelque sortent le lien entre ces différentes orientations.

Ces conclusions résument les observations sur les visualisations générées, tentent d'en trouver des explications qui nous permettent d'oser une généralisation, mais peuvent difficilement dépasser en précision les visualisations elles-mêmes. C'est pourquoi le lecteur est fortement invité à les tester par lui-même (et qui sait, peut-être trouver de tendances que l'on aurait manquée).

# 5 Conclusion, travaux inachevés et ouverture

L'objectif principal était de montrer ce que le formalisme des SBN permettait. Et à cet égard le présent travail semble nous indiquer qu'il parvient assez bien à donner des interprétations claires de concepts généraux (comme la pluripotence), et qu'il est suffisamment pratique et riche pour que l'exploration de ses possibilités soit faisable et porteuse de sens.

Travaux inachevés (évoqués au fil du papier, mais rassemblés ici) :
- Affichage d'arbre généalogique issu des simulations.
- Minimisation de poids.
- PluripotentSignedBooleanNetworkInhibiteur.
- Régression linéaire pour confirmer Wagner.
- Exploration dirigée (MCSBN).
- Meilleure FrontEnd (toujours améliorable).

Idée futures :

- Gestion des arêtes non existantes (et pas juste de poids nuls)

- Comparaison des dimensions entre elles.

- Explorer d'autres aspects de la pluripotence : noeuds contrôlés, offuscation et vecteurs de décomposition partiels.

- Exploitation plus aboutie des résultats.

# Références


[1] Nicolas Glade. "Le Vivant Rare, Faible et Amorphe — Évolution depuis les Origines jusqu'à la Vie telle qu'elle nous Apparaît". HAL : tel-04075075. Habilitation à Diriger des Recherches. Université Grenoble Alpes, 2022.

[2] Rémi Segretain. "Complexité et évolution : étude des Réseaux Booléens à Signe". École doctorale ISCE, spécialité MBS. Directeur : Nicolas Glade. Thèse de doctorat. Université Grenoble Alpes, 2023.

[3] Rémi Segretain et al. "Implementation of a Computing Pipeline for Evaluating the Extensibility of Boolean Networks' Structure and Function". In : (2020). bioRxiv preprint, doi :10.1101/2020.10.02.323949.

[4] Ai-Ling Nguyen Bonnet. *Évolution des réseaux booléens à signe*. Rapport de stage M1. Encadrants : Sergiu Ivanov et Nicolas Glade. Université Grenoble Alpes, 2023.

[5] Rémy Defossez. *Le mille-feuille évolutionnaire, une nouvelle formalisation de l'évolution*. Rapport de stage L3. Encadrants : Sergiu Ivanov et Nicolas Glade. ENS Paris Ulm / Université Grenoble Alpes, 2021.

[6] Danwen Wu. *Formalization, identification and decomposition of pluripotents Boolean networks*. Rapport de stage M1. Encadrants : Nicolas Glade et Laurent Trilling, TIMC Laboratory. Université Grenoble Alpes, 2024.

[7] Andreas Wagner. "Robustness and evolvability : a paradox resolved". In : *Proceedings of the Royal Society B* 275.1630 (2008), p. 91-100. doi : 10.1098/rspb.2007.1137.

[8] Mutsunori Banbara et al. "Clingcon : The Next Generation". In : *Theory and Practice of Logic Programming* (2017). arXiv :1705.04569.

[9] Mohimenul Kabir et al. "ApproxASP — a Scalable Approximate Answer Set Counter". In : *Proceedings of the Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI-22)*. 2022, p. 5755-5764.

[10] Minfeng Zhu et al. "DRGraph : An Efficient Graph Layout Algorithm for Large-scale Graphs by Dimensionality Reduction". In : (2020). arXiv :2008.07799.

# A Guide d'utilisation des programmes

Cette annexe explique comment utiliser les programmes au point, avant tout la pipeline JPP, et dans une moindre mesure MCSBN. Ses deux premières parties réunissent ce qu'il faut savoir avant de s'en servir, à savoir où ils se trouvent dans le dépôt, comment installer leurs dépendances, et sous quel format ils écrivent les CSV que lisent les visualisations, tandis que les suivantes détaillent l'usage de chacun d'eux.

## A.1 Organisation du dépôt et installation

La figure 5 montre quel programme produit quoi et qui le lit, et le tableau 3 indique où se trouve chacun d'eux dans le dépôt, ainsi que la partie de cette annexe qui en détaille l'usage.

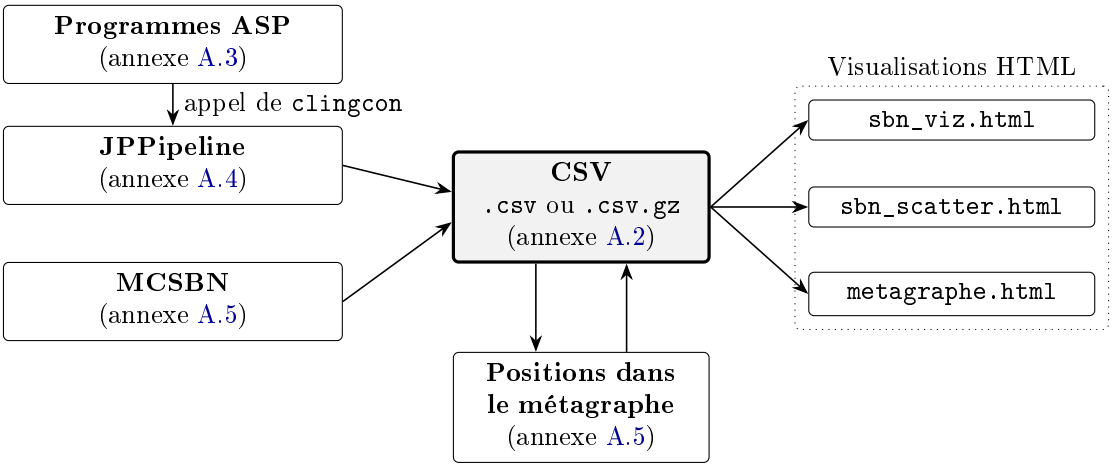



FIGURE 5 – Chaîne de traitement. Le calcul des positions dans le métagraphe est facultatif : il relit un CSV et le réécrit avec deux colonnes de plus.

JPP et MCSBN écrivent des CSV au même format, à la colonne `tree` près, que l'on peut compléter par les positions des dynamiques dans le métagraphe avant de les ouvrir dans les pages HTML, lesquelles fonctionnent directement dans le navigateur et lisent aussi les fichiers compressés en gzip. Le dépôt ne contient pas ces CSV, que le `.gitignore` exclut comme les binaires compilés, si bien qu'il faut les régénérer ou se les procurer à part.

Le fichier `environment.yml`, à la racine du dépôt, décrit l'environnement conda du projet, qui réunit les dépendances peu courantes (`clingcon`, les bibliothèques GSL et Boost qu'utilise DRGraph, et celles des scripts Python) ; on le crée puis l'active depuis la racine, ce qui met `clingcon` dans le `PATH`, où JPP va le chercher :

```
conda env create -p .conda -f environment.yml && conda activate ./.conda
```

Une passe complète consiste alors à lancer JPP pour chaque couple (programme ASP, dimension) puis MCSBN, à compresser les gros CSV avec `gzip` et à ajouter au besoin les

| Partie | Emplacement | Contenu |
|---|---|---|
| A.3. Programmes ASP | `JPPipeline/src/asp/` | Programmes d'énumération et leurs fichiers d'appui (`setup_btree_{1..6}d.lp`, `Subdivision.lp`). |
| A.4. JPPipeline | `JPPipeline/src/java_src/`, `JPPipeline/lib/jpp.jar`, `.vscode/launch.json` | Pipeline Java, bibliothèque JPP [À VÉRIFIER : auteur et dépôt d'origine] et configurations de lancement ; les CSV sont écrits dans `JPPipeline/out/`. |
| A.5. MCSBN | `MCSBN/` | Générateur Monte-Carlo en C et en Python, et placement du métagraphe par DRGraph (`tools/DRGraph/`). |
| Visualisations | `html_visualization_lib/`, `*.html` à la racine | Sources des trois pages, et leurs versions autonomes, à régénérer avec `build_standalone.py` après toute modification des sources. |
| A.6. Scripts annexes | `SBN_pluripotent/`, `SBN_pluripotent_Laurent/`, `JPPipeline/src/python/` | Calcul des bornes des poids, versions antérieures des programmes et post-traitements abandonnés. |

TABLE 3 – Emplacement dans le dépôt des programmes décrits dans chaque partie de cette annexe.

positions du métagraphe.

Le dépôt contient bien d'autres programmes que ceux référencés ici (essais, versions antérieures, outils abandonnés en cours de route), et rien de ce qui n'est pas présenté comme fonctionnel dans cette annexe n'est garanti sans s'être penché sur le code pour le comprendre. Il en va de même des données produites avant la rédaction de cette annexe, dont certaines suivent d'anciennes conventions (noms de colonnes, fonctions écrites en entier, autre règle de mutation) et qu'il vaut mieux régénérer que comparer aux nouvelles.

## A.2 Format des CSV

Chaque CSV commence par une ligne d'en-tête, et comme les pages HTML repèrent les colonnes par leur nom, des colonnes en plus ne les gênent pas. Le tableau 4 donne ces colonnes dans l'ordre où JPP les écrit, MCSBN écrivant les mêmes à l'exception de `tree`.

Les états sont numérotés de 0 à $2^d - 1$, le bit $i - 1$ de l'état $s$ valant 1 quand le nœud $i$ est actif (le nœud 1 est donc le bit de poids faible), et le caractère numéro $s$ de `f_`$j$, en comptant depuis 0 à partir de la gauche, donne la valeur du nœud $j$ dans l'état qui suit $s$. Les poids sont des entiers compris entre $-w_\mathrm{b}(d)$ et $w_\mathrm{b}(d)$, avec $w_\mathrm{b}(d) = 1$, 1, 2, 3, 5 et 6 pour $d$ allant de 1 à 6 (en $d = 6$, cette valeur diffère de la borne du tableau 1).

La colonne `tree` écrit l'arbre de décomposition de façon imbriquée, en notant `*` une feuille (un sous-cube qui n'est plus découpé) et `N[A,B]` une coupe selon le nœud de contrôle `N`, où `A` est l'arbre de la moitié où `N` est inactif et `B` celui de la moitié où il est actif. Ainsi `1[2[*,*],*]` coupe selon le nœud 1 puis recoupe selon le nœud 2 la seule moitié où le nœud 1 est inactif, et l'on retrouve le vecteur en comptant les feuilles, $v_k$ étant le nombre

| Colonne | Contenu |
|---|---|
| `v_`$d$, …, `v_0` | Vecteur de décomposition $\langle v_d, \ldots, v_0 \rangle$, où $v_k$ est le nombre de sous-cubes (hyper-faces) de dimension $k$ aux feuilles de l'arbre. |
| `tree` | Arbre de décomposition étiqueté par les nœuds de contrôle, entre guillemets (notation ci-dessous) ; vide si le programme ASP ne le fournit pas. |
| `f_1`, …, `f_`$d$ | Fonction de transition du nœud $j$, en table de vérité de $2^d$ caractères `0` ou `1`. |
| `"w_`$i$`,`$j$`"` | Poids de l'arc $i \to j$. |
| `CycleLenMSQ` | Moyenne, sur les attracteurs, du carré de leur longueur. |
| `NumAttractors` | Nombre d'attracteurs, points fixes et cycles. |
| `GenotypeCount` | Nombre de matrices de poids qui réalisent la même dynamique. |
| `Robustness_std`, `Robustness_mean` | Écart-type et moyenne de la robustesse de ces matrices. |
| `Evolvability` | Évolutivité de la dynamique. |

TABLE 4 – Colonnes des CSV, dans l'ordre du fichier ($d$ est la dimension). Les statistiques sont définies dans la section « Statistiques, propriétés, métriques ».

de feuilles à la profondeur $d - k$.

Une ligne ne correspond pas à un réseau mais à un couple (dynamique, arbre de décomposition), car les programmes `undercontrol` produisent une même dynamique une fois par arbre qu'elle vérifie, et le vecteur trivial $\langle 1, 0, \ldots, 0 \rangle$, qui n'impose aucune contrainte, les contient toutes ; le CSV $d = 3$ de `PSBN_undercontrol_taggedtree` compte ainsi 34 343 lignes pour seulement 32 768 dynamiques. Les statistiques d'une dynamique sont les mêmes sur toutes ses lignes, alors que ses poids peuvent varier d'une ligne à l'autre, les colonnes `w_*` ne donnant qu'une matrice parmi toutes celles qui la réalisent ; pour compter des dynamiques, il faut donc dédoublonner selon $(f_1, \ldots, f_d)$, comme le fait `metagraphe.html`.

Le calcul des positions dans le métagraphe ajoute deux colonnes en fin de ligne, `metagraph_x` et `metagraph_y`, que `metagraphe.html` utilise pour placer les dynamiques. À l'inverse, JPP peut omettre les colonnes `f_*` et `w_*` (dernier argument à 0), mais seule `sbn_scatter.html` peut alors exploiter le fichier. Les pages HTML lisent par ailleurs la dimension dans le préfixe du nom de fichier (`2d_`, `3d_`…), qu'il vaut donc mieux conserver.

## A.3 Programmes ASP d'énumération

La version aboutie des programmes ASP est `PSBN_undercontrol_taggedtree.lp`, dans `JPPipeline/src/asp/`, ou sa variante `PTBN_undercontrol_taggedtree.lp` si l'on admet des sous-réseaux qui ne sont pas des SBN. Les autres programmes du dossier (familles `undercontrol`, `treeinsolver` et `undecomposablePBN_beyond`) sont des versions intermédiaires, dont l'objectif a été atteint ou non selon les cas (section « Correction »), mais qui n'ont pas été maintenues jusqu'au bout.

Le programme lit la dimension et le vecteur de décomposition dans des constantes, et

se lance seul avec

```
clingcon <n> --project -c d=<d> -c v<d>=<v_d> ... -c v0=<v_0> \
      JPPipeline/src/asp/PSBN_undercontrol_taggedtree.lp
```

`<n>` nombre de solutions à afficher (0 pour toutes) ;

`d` dimension, au plus 5 ;

`v<k>` nombre de sous-cubes de dimension $k$ du vecteur de décomposition ; laissées à $-1$, leur valeur par défaut, ces constantes n'imposent aucun vecteur et le programme énumère tous les SBN de dimension $d$.

Chaque solution donne les atomes `transition_function_bit(N,S)`, présents quand le nœud `N` est actif dans l'état qui suit `S`, les poids `w(I,J)` de l'arc `I` → `J`, et les atomes `branch_node/4` et `leaf_node/3`, qui décrivent l'arbre de décomposition. C'est ce format que lit JPP (`ClingconUtil.java`), si bien qu'un nouveau programme doit le respecter pour être utilisé par la chaîne.

## A.4 JPPipeline : énumération parallèle et statistiques

JPP se compile et se lance depuis la racine du dépôt, avec `clingcon` dans le `PATH`, que fournit par exemple l'environnement conda du projet (annexe A.1) :

```
javac -cp JPPipeline/lib/jpp.jar -d JPPipeline/bin \
      $(find JPPipeline/src/java_src/PSBN -name '*.java')
java -cp JPPipeline/bin:JPPipeline/lib/jpp.jar PSBN.Main \
      <dim> <lp_enumeratorFile> <setup_btreeFolder> <nSolutions> \
      <output_path> [includeWeightsAndTransitions]
```

`<dim>` dimension $d$ des réseaux ;

`<lp_enumeratorFile>` programme ASP qui énumère les réseaux (annexe A.3) ;

`<setup_btreeFolder>` dossier contenant `setup_btree_<d>d.lp` et `Subdivision.lp`, en pratique `JPPipeline/src/asp` ;

`<nSolutions>` nombre maximal de solutions par vecteur de décomposition (0 pour toutes) ;

`<output_path>` dossier où écrire le CSV ;

`[includeWeightsAndTransitions]` facultatif, 0 pour omettre les colonnes `f_*` et `w_*` (1 par défaut).

Par exemple, la configuration `Main` de `.vscode/launch.json`, qu'on peut aussi lancer directement depuis VS Code, correspond à

```
java -cp JPPipeline/bin:JPPipeline/lib/jpp.jar PSBN.Main 2 \
      JPPipeline/src/asp/PSBN_undercontrol_taggedtree.lp JPPipeline/src/asp \
      0 JPPipeline/out 1
```

Le CSV, nommé `<dim>d_<programme>_output.csv`, s'écrit au fur et à mesure dans le dossier `<output_path>`, et il vaut mieux le compresser ensuite avec `gzip`, puisque les pages HTML lisent directement les fichiers `.csv.gz`. Pour suivre un long calcul, les options `-Djpp_monitoring=true` et `-Djpp_monitoring.stepTime=5`, placées avant `-cp` (elles figurent déjà dans `launch.json`), affichent toutes les cinq secondes l'avancement de chaque étape et la mémoire utilisée.

L'argument `<nSolutions>` permet de ne demander à `clingcon` qu'un nombre limité de solutions par vecteur. Comme `clingcon` est lancé avec une graine fixe, cette énumération partielle redonne les mêmes réseaux d'une exécution à l'autre. Lorsque ce nombre est nul ou dépasse 1000, un garde-fou arrête l'énumération d'un vecteur à 1000 solutions et la relance en tranches, en fixant les premiers bits de la fonction du nœud 1 grâce à `Subdivision.lp`, pour limiter la mémoire de chaque appel ; le nombre de solutions demandé est alors réparti entre les tranches. Ce seuil de 1000 solutions (`SUBDIVISION_THRESHOLD`) et le nombre maximal d'énumérations non subdivisées menées en parallèle (10, par `setMaximumParallelTasks`) sont codés en dur dans `Pipe.java`, où il faut les modifier pour les adapter à la machine, les autres étapes utilisant déjà autant de fils d'exécution que la machine a de cœurs. Limiter ainsi le nombre d'énumérations menées en parallèle évite d'engorger JPP.

Pour ajouter une statistique, on la calcule dans la méthode `process` de `Statifier.java`, qui reçoit chaque réseau et renvoie sa ligne de CSV, puis on ajoute son nom, à la bonne place, dans l'en-tête que construit la méthode `reset` de `Pipe.java` ; les statistiques qui portent sur les attracteurs sont, elles, calculées dans `SBN.toCsvRow`.

## A.5 MCSBN : exploration Monte-Carlo

MCSBN se compile et se lance depuis son dossier, `MCSBN/`, où il garde en cache, dans `.cache/`, les tables qu'il calcule à la première exécution dans chaque dimension :

```
make && make metalayout && make drgraph
```

Ces trois cibles produisent respectivement le générateur `bin/mcsbn`, le calcul des positions dans le métagraphe `bin/metagraph_layout` et le binaire `tools/DRGraph/Vis` de DRGraph, qui va chercher GSL et Boost dans l'environnement conda (annexe A.1) ; le `Makefile` fonctionne aussi sous Windows, depuis le shell MSYS2 MinGW64.

Le générateur s'utilise ainsi :

```
./bin/mcsbn -d <d> [-n <n>] [-o <fichier>] [--measure variety|genotype]
      [--seed <s>] [--max-draws <m>] [--exhaustive] [--no-weights]
      [--progress|--no-progress]
```

`-d <d>` dimension, de 1 à 6 ;

`-n <n>` nombre de dynamiques distinctes à produire (100 000 par défaut) ;

`-o <fichier>` fichier de sortie, compressé si son nom finit par `.gz` (sortie standard par défaut) ;

`--measure variety` (par défaut) tire chaque fonction uniformément parmi les SBF distinctes, si bien que toutes les dynamiques sont équiprobables, alors que `genotype` la tire en proportion de son nombre de réalisations, comme un tirage uniforme des matrices de poids ;

`--seed <s>` graine du tirage, fixe par défaut, de sorte que deux exécutions identiques donnent le même fichier ;

`--max-draws <m>` nombre maximal de tirages, au-delà duquel le programme s'arrête même s'il n'a pas obtenu $n$ dynamiques distinctes, ce qui n'arrive qu'en petite dimension ($50n$ par défaut) ;

`--exhaustive` énumère toutes les dynamiques au lieu de tirer, ce qui n'est accepté qu'en deçà de $5 \cdot 10^7$ dynamiques ;

`--no-weights` omet les colonnes `f_*` et `w_*` ;

`--progress, --no-progress` force ou coupe l'affichage de l'avancement, actif par défaut dans un terminal.

La version Python, `src/python/mcsbn.py`, accepte les mêmes options mais tourne bien plus lentement ; elle sert de référence aux tests.

Un tirage de `mcsbn` est trop clairsemé pour que le métagraphe ait des arêtes, et c'est `src/python/core_then_walk.py` qui produit un paysage où il en a, en gardant toutes les dynamiques les plus décomposées puis en y ajoutant leur voisinage :

```
python3 src/python/core_then_walk.py -d <d> [-M <m>] [-n <n>] [-o <fichier>]
    [--min-leaves <k>] [--max-free <k>] [--fanout <k>]
    [--measure variety|genotype] [--seed <s>] [--no-weights]
```

`-M <m>` nombre maximal de dynamiques du cœur décomposé (3000 par défaut) ;

`-n <n>` nombre total de dynamiques, cœur compris (30 000 par défaut) ;

`--min-leaves <k>` nombre minimal de feuilles des arbres du cœur, à augmenter pour un cœur plus petit et plus décomposé (2 si $d \leq 3$, 8 sinon) ;

`--max-free <k>` nombre maximal de nœuds libres par arbre du cœur, à augmenter pour y inclure des dynamiques moins décomposées, au prix du temps de calcul (illimité si $d \leq 3$, 0 sinon) ;

`--fanout <k>` nombre de nouveaux voisins pris par dynamique à chaque étape de l'exploration du voisinage (4 par défaut).

Les autres options sont celles de `mcsbn`, que le script utilise s'il est compilé (et sinon une version Python plus lente).

Les positions dans le métagraphe s'ajoutent à n'importe quel CSV, de MCSBN comme de JPP, avec

```
python3 src/python/add_metagraph_layout.py <entrée> [-o <sortie>] [--vis <Vis>]
      [--neg <k>] [--samples <k>] [--gamma <x>] [--mode <k>] [--A <x>] [--B <x>]
```

où les dernières options sont celles de DRGraph. Par défaut, la sortie s'appelle `<entrée>_metagraph.csv` (ou `.csv.gz`) et `Vis` est cherché dans `tools/DRGraph/`. Le portage en C, `bin/metagraph_layout`, prend les mêmes arguments et va plus vite sur les gros fichiers, mais se limite à $d \leq 4$ ; son option `--stats-only` n'écrit rien et affiche seulement les composantes connexes du métagraphe.

Enfin, `make test` compare MCSBN aux CSV que JPP produit avec les programmes `treeinsolver` (`<d>d_PSBN_undercontrol_treeinsolver_output.csv` et sa version `PTBN`, dans `JPPipeline/out/`), qu'il faut donc avoir générés. Les autres scripts de `src/python/` sont des modules communs ou des outils intermédiaires.

## A.6 Scripts annexes et programmes historiques

Pour ce qui est des autres scripts mis au point, ils se limitent au calcul de la borne des poids et à la génération des fichiers `setup_btree`. La borne des poids $w_{\mathrm{b}}(n)$ et le nombre d'orbites de SBF du tableau 1 se calculent avec

```
python3 SBN_pluripotent/w_bound_orbits.py [--max-n <n>] [--big-m <m>]
```

- `--max-n <n>` dimension maximale traitée (5 par défaut), le script étant conçu pour aller jusqu'à 6 ;
- `--big-m <m>` borne supérieure des poids dans le programme linéaire en nombres entiers (200 par défaut, largement suffisant).

Le script affiche, pour chaque dimension, la borne, le nombre d'orbites et une SBF qui l'atteint.

Les fichiers `setup_btree_{1..6}d.lp`, déjà fournis dans `JPPipeline/src/asp/`, se régénèrent dans le dossier courant avec

```
python3 SBN_pluripotent/gen_setup_btree.py <d> [<d> ...]
```

Le reste relève des étapes antérieures du stage, à savoir les programmes de Laurent et leurs scripts d'export (`SBN_pluripotent_Laurent/`), l'export `export_btree.py` et le comptage de fréquences par ApproxASP (`JPPipeline/src/asp/freq_counting/`), que remplace le calcul exact de `GenotypeCount` dans JPP.